

make notes for me out of this transcript , explain everything properly
deeply

SQL Complete Course Notes (Hindi Transcript)

Introduction

This is a complete SQL course covering:

- Database fundamentals
- PostgreSQL installation (Windows & Mac)
- CRUD operations
- Data types & Constraints
- Clauses & Operators
- Aggregation functions
- String functions
- ALTER & CASE
- Relationships & Joins
- Views & Procedures

1. Database Basics

What is a Database?

- A collection of organized data that can be easily accessed, managed, and updated.
- Think of it as a digital version of a well-organized notebook.
- Data is stored electronically in **tables** (like Excel sheets) with rows and columns.
- You can query data using a language called **SQL**.

Example:

Students Table:

Student ID	Name	Age	Grade
1	Akarsh	23	A
2	Anjali	22	B
3	Raj	23	C

You can ask the database:

- "Show all students with grade A" → Returns Akarsh & Raj
- "Add a new student" → Inserts a new row
- "Update Anjali's age" → Updates the age column
- "Delete Raj's record" → Removes the row

What is RDBMS?

Relational Database Management System — Software that allows you to create, read, update, and delete data using SQL.

Breakdown:

- **Relational** → Data is organized into tables that are related to each other
- **Database** → Structured way to store data
- **Management System** → Software that helps you manage it

Features:

- Data stored in tables (rows & columns)
- Relationships between tables
- Supports complex queries, joins, CTEs, window functions
- Highly extensible
- ACID compliant

SQL vs NoSQL

Feature	SQL	NoSQL
Structure	Structured (tables)	Unstructured (objects/documents)
Example	PostgreSQL, MySQL, SQLite, Oracle	MongoDB
Query Language	SQL	Varies
Best for	Complex relationships, large structured data	Flexible schemas, unstructured data

Why PostgreSQL?

- Open source (free)
- Most feature-rich and powerful
- Supports complex queries, joins, CTEs, window functions
- Highly extensible (custom data types, operators)
- Supports NoSQL features (JSON)
- Used for enterprise-grade applications
- 30+ years old but still widely used

RDBMS Examples:

- PostgreSQL ← We are learning this
- MySQL
- SQLite (built into Django)
- Oracle
- SQL Server

Why not Excel?

- Excel struggles with big data (100,000+ rows)
- Searching, updating, and managing large datasets becomes slow

- SQL databases handle millions of rows efficiently
- Better consistency, reliability, and security

2. PostgreSQL Installation

Windows Installation:

1. Go to browser → Search "PostgreSQL download"
2. Click official website
3. Choose Windows → Download the installer (latest version 17.5)
4. Run installer → Next → Next → Next
5. Keep all checkboxes ticked (SQL Server, PG Admin, Stack Builder, Command Line Tools)
6. Set a password (e.g., 1234) when prompted
7. Wait for installation (1–1.5 hours)
8. Uncheck "Launch Stack Builder" → Finish
9. Search "PG Admin" → Open PG Admin 4
10. Double-click Servers → Enter password
11. You can now create databases and run queries

Mac Installation:

1. Go to browser → Search "PostgreSQL download"
2. Click official website
3. Choose Mac OS → Download installer (DMG file, ~300 MB)
4. Open installer → Enter Mac password
5. Next → Next → Next
6. Keep all checkboxes ticked
7. Set PostgreSQL password (e.g., 1234)
8. Uncheck "Launch Stack Builder" → Finish
9. Search "PG Admin" → Open PG Admin 4
10. Double-click Servers → Enter password

Mac Command Line Setup:

- Search "SQL Shell" → Open it
- Type `psql -U postgres`
- If "command not found", add PostgreSQL path to Z profile:
 - Open terminal → Type `sudo find / -name psql` to find path
 - Use ChatGPT to identify correct path
 - Add path to `~/.zshrc` using `echo 'export PATH="/path/to/psql:$PATH"' >> ~/.zshrc`
 - Run `source ~/.zshrc`
 - Verify with `psql --version`
- Connect: `psql -U postgres`
- List databases: `\l`
- Connect to database: `\c database_name`

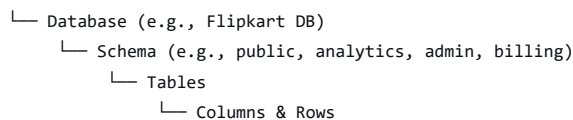
Key Commands:

- `\l` — List all databases
- `\c db_name` — Connect to a database
- `\d table_name` — Describe table structure
- `\dt` — List all tables
- `\dn` — List all schemas
- `\q` — Quit

3. PostgreSQL Structure

text

Server



- **Database** → Big container that stores all your data. Multiple databases can exist in one server.
- **Schema** → Like a folder inside a database. Organizes tables, views, functions. Default schema is `public`.
- **Table** → Main place where data is stored (rows & columns).
- **Columns** → Define what type of data is stored (name, age, etc.)
- **Rows** → Individual records

Example (LinkedIn):

- Database: LinkedIn DB
- Schemas: `public`, `analytics`, `admin`, `billing`
- Tables in `public`: `users`, `posts`, `connections`, `messages`
- Tables in `analytics`: `purchases`, `impressions`, `likes`
- Tables in `admin`: `user_profiles`, `admin_data`
- Tables in `billing`: `payments`, `subscriptions`

4. CRUD Operations

CREATE — Create Database

sql

```
CREATE DATABASE new_db;
```

CREATE — Create Table

sql

```

CREATE TABLE students (
  student_id SERIAL PRIMARY KEY,
  name VARCHAR(50),

```

```
    age INT,  
    grade CHAR(1)  
);
```

INSERT — Add Data

```
sql  
  
INSERT INTO students (name, age, grade)  
VALUES ('Akarsh', 23, 'A'),  
       ('Anjali', 22, 'B'),  
       ('Raj', 23, 'C');
```

READ — Select Data

```
sql  
  
SELECT * FROM students;  
SELECT name FROM students;  
SELECT name, age FROM students WHERE age = 23;
```

UPDATE — Modify Data

```
sql  
  
UPDATE students  
SET age = 24  
WHERE name = 'Akarsh';
```

DELETE — Remove Data

```
sql  
  
DELETE FROM students  
WHERE name = 'Raj';
```

Common Problems Without Constraints:

- NULL values can appear (e.g., student_id missing)
- Duplicate values can exist (e.g., two students with same ID)
- Wrong data types can be entered
- No validation on data

5. Data Types

Numeric Types:

Type	Size	Range	Use Case
SMALLINT	2 bytes	-32,768 to 32,767	Age, small nu
INT/INTEGER	4 bytes	-2B to 2B	Quantity, ger
BIGINT	8 bytes	-9Q to 9Q	Billions+, larç

Type	Size	Range	Use Case
DECIMAL/NUMERIC(p,s)	Variable	p digits total, s after decimal	Prices, exact
REAL	4 bytes	6 decimal digits precision	Ratings, appi
DOUBLE PRECISION	8 bytes	15 decimal digits precision	Scientific cal
SERIAL	4 bytes	Auto-incrementing integer	Primary keys

Example:

```
sql

CREATE TABLE numbers (
  id SERIAL PRIMARY KEY,
  age SMALLINT,
  price NUMERIC(8,2), -- 8 total digits, 2 after decimal
  rating REAL
);
```

- NUMERIC(8,2) → Max 8 digits total, 2 after decimal. E.g., 123456.78
- CHAR(5) → Fixed length, pads with spaces if shorter
- VARCHAR(5) → Variable length, no padding
- TEXT → Unlimited length

Character Types:

Type	Description
CHAR(n)	Fixed-length string, pads with spaces
VARCHAR(n)	Variable-length string with max limit
TEXT	Unlimited length string

Example:

```
sql

CREATE TABLE strings (
  code CHAR(5), -- Always 5 characters
  email VARCHAR(100), -- Max 100 characters
  bio TEXT -- Unlimited
);
```

Boolean Type:

```
sql

is_active BOOLEAN DEFAULT TRUE
```

- Values: TRUE, FALSE, NULL

Date & Time Types:

Type	Description	Example
DATE	Only date	2025-07-15

Type	Description	Example
TIME	Only time	14:30:00
TIMESTAMP	Date + time	2025-07-15 14:30:00
TIMESTAMPTZ	Date + time + timezone	2025-07-15 14:30:00+05:30
INTERVAL	Time duration	1 hour 30 minutes

Example:

```
sql

created_at DATE DEFAULT CURRENT_DATE,
last_updated TIMESTAMP DEFAULT NOW()
```

6. Constraints

Constraints are rules enforced on columns to maintain data integrity.

PRIMARY KEY:

- Uniquely identifies each row
- Cannot be NULL
- Must be unique
- Usually an ID column

```
sql

student_id SERIAL PRIMARY KEY
```

NOT NULL:

- Column must have a value
- Cannot be left empty

```
sql

name VARCHAR(100) NOT NULL
```

UNIQUE:

- All values in column must be different
- Can have NULL (unless NOT NULL also applied)

```
sql

email VARCHAR(100) UNIQUE NOT NULL
```

DEFAULT:

- Sets a default value if none provided

```
sql
```

```
created_at DATE DEFAULT CURRENT_DATE,  
stock_quantity INT DEFAULT 0
```

CHECK:

- Validates data based on a condition

```
sql  
  
age INT CHECK (age >= 18),  
price NUMERIC CHECK (price >= 0),  
stock_quantity INT CHECK (stock_quantity >= 0)
```

FOREIGN KEY:

- Links two tables together
- References PRIMARY KEY of another table
- Ensures referential integrity

```
sql  
  
FOREIGN KEY (student_id) REFERENCES students(student_id)
```

Important Rule:

- Foreign key values must exist in the primary key table
- If primary key has 1,2,3 and foreign key has 1,2,3,4 → Error (4 doesn't exist)
- If primary key has 1,2,3,4 and foreign key has 1,2,3,4 → Works fine

Multiple Constraints Together:

```
sql  
  
email VARCHAR(100) UNIQUE NOT NULL,  
age INT CHECK (age >= 18)
```

7. ALTER TABLE

Used to modify existing table structure.

Add Column:

```
sql  
  
ALTER TABLE students  
ADD COLUMN email VARCHAR(100) DEFAULT 'Not Provided';
```

Drop Column:

```
sql  
  
ALTER TABLE students  
DROP COLUMN email;
```

Rename Column:

sql

```
ALTER TABLE students  
RENAME COLUMN name TO full_name;
```

Change Data Type:

sql

```
ALTER TABLE students  
ALTER COLUMN age TYPE SMALLINT;
```

Set Default Value:

sql

```
ALTER TABLE students  
ALTER COLUMN age SET DEFAULT 18;
```

Remove Default Value:

sql

```
ALTER TABLE students  
ALTER COLUMN age DROP DEFAULT;
```

Add Constraint:

sql

```
ALTER TABLE students  
ADD CONSTRAINT age_check CHECK (age >= 0);
```

Drop Constraint:

sql

```
ALTER TABLE students  
DROP CONSTRAINT age_check;
```

Rename Table:

sql

```
ALTER TABLE students  
RENAME TO school_students;
```

8. SQL Clauses

Clauses are building blocks of SQL queries.

Clause	Purpose
SELECT	Choose columns to display
FROM	Specify table
WHERE	Filter rows based on condition
GROUP BY	Group rows for aggregation
HAVING	Filter aggregated groups
ORDER BY	Sort results (ASC/DESC)
LIMIT	Limit number of rows
AS	Rename column/table temporarily
DISTINCT	Return only unique values

Examples:

SELECT & FROM:

sql

```
SELECT name, price FROM products;
```

WHERE:

sql

```
SELECT * FROM products WHERE category = 'Electronics';
```

GROUP BY:

sql

```
SELECT category, COUNT(*) FROM products GROUP BY category;
```

HAVING:

sql

```
SELECT category, COUNT(*) FROM products  
GROUP BY category  
HAVING COUNT(*) > 1;
```

ORDER BY:

sql

```
SELECT * FROM products ORDER BY price ASC;  -- Low to High  
SELECT * FROM products ORDER BY price DESC; -- High to Low
```

LIMIT:

sql

```
SELECT * FROM products LIMIT 3;
```

AS (Alias):

```
sql

SELECT name AS item_name, price AS item_price FROM products;
```

DISTINCT:

```
sql

SELECT DISTINCT category FROM products;
```

9. Operators

Comparison Operators:

Operator	Meaning
=	Equal to
!= or <>	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

Examples:

```
sql

SELECT * FROM products WHERE price > 1000;
SELECT * FROM products WHERE category != 'Electronics';
SELECT * FROM products WHERE price BETWEEN 400 AND 1000;
SELECT * FROM products WHERE category IN ('Electronics', 'Fitness', 'Home & Kitchen');
```

Logical Operators:

Operator	Meaning
AND	All conditions must be true
OR	Any condition can be true
NOT	Reverses the condition

Examples:

```
sql

-- AND
SELECT * FROM products WHERE price < 1000 AND category = 'Electronics';

-- OR
SELECT * FROM products WHERE category = 'Electronics' OR category = 'Fitness';
```

```
-- NOT
SELECT * FROM products WHERE NOT category = 'Electronics';
```

Pattern Matching (LIKE):

Pattern	Meaning
LIKE 'W%'	Starts with W
LIKE '%W'	Ends with W
LIKE '%W%'	Contains W
LIKE '_W%'	Second letter is W
LIKE '__W%'	Third letter is W

Examples:

```
sql

SELECT * FROM products WHERE sku_code LIKE 'W%';      -- Starts with W
SELECT * FROM products WHERE sku_code LIKE '%123%';    -- Contains 123
SELECT * FROM products WHERE sku_code LIKE '_B%';      -- Second letter is B
```

NULL Handling:

```
sql

SELECT * FROM products WHERE column IS NULL;
SELECT * FROM products WHERE column IS NOT NULL;
```

10. Aggregation Functions

Used to summarize data.

Function	Purpose
COUNT()	Count number of rows
SUM()	Add numeric values
AVG()	Calculate average
MIN()	Find smallest value
MAX()	Find largest value
ROUND()	Round to specified decimals

Examples:

```
sql

-- Count total products
SELECT COUNT(product_id) FROM products;

-- Sum of all prices
SELECT SUM(price) FROM products;
```

```
-- Average price
SELECT AVG(price) FROM products;
SELECT ROUND(AVG(price), 2) FROM products;

-- Minimum price
SELECT MIN(price) FROM products;

-- Maximum price
SELECT MAX(price) FROM products;

-- Sum with condition
SELECT SUM(price) FROM products WHERE category = 'Electronics';

-- Group by with aggregation
SELECT category, MAX(price) FROM products GROUP BY category;
```

11. String Functions

Function	Purpose	Example
UPPER()	Convert to uppercase	UPPER(name)
LOWER()	Convert to lowercase	LOWER(name)
LENGTH()	Length of string	LENGTH(sku_code)
SUBSTRING()	Extract part of string	SUBSTRING(sku_code, 1, 2)
LEFT()	Extract from left	LEFT(sku_code, 2)
RIGHT()	Extract from right	RIGHT(sku_code, 2)
CONCAT()	Join strings	CONCAT(name, category)
CONCAT_WS()	Join with separator	CONCAT_WS(' ', name, category)
TRIM()	Remove spaces	TRIM(' hello ')
REPLACE()	Replace text	REPLACE(sku_code, 'WM', 'GG')

Examples:

```
sql

SELECT UPPER(name) FROM products;
SELECT LOWER(name) FROM products;
SELECT LENGTH(sku_code) FROM products;
SELECT SUBSTRING(sku_code, 1, 2) FROM products;
SELECT LEFT(sku_code, 2) FROM products;
SELECT RIGHT(sku_code, 2) FROM products;
SELECT CONCAT(name, ' ', category) FROM products;
SELECT CONCAT_WS(' ', name, category, sku_code) FROM products;
SELECT TRIM(' hello ');
SELECT REPLACE(sku_code, 'WM', 'GG') FROM products;
```

12. CASE Statements

Conditional expressions (like IF-ELSE).

Syntax:

```
sql

SELECT column1,
       CASE
         WHEN condition1 THEN result1
         WHEN condition2 THEN result2
         ELSE result3
       END AS new_column
FROM table_name;
```

Example 1: Price Tag

```
sql

SELECT name, price,
       CASE
         WHEN price > 1000 THEN 'Expensive'
         WHEN price BETWEEN 500 AND 1000 THEN 'Moderate'
         ELSE 'Cheap'
       END AS price_tag
FROM products;
```

Example 2: Availability Status

```
sql

SELECT name,
       CASE
         WHEN is_available THEN 'In Stock'
         ELSE 'Out of Stock'
       END AS stock_status
FROM products;
```

Example 3: Stock Level

```
sql

SELECT name,
       CASE
         WHEN stock_quantity > 100 THEN 'High Stock'
         WHEN stock_quantity BETWEEN 30 AND 100 THEN 'Medium Stock'
         ELSE 'Low Stock'
       END AS stock_level
FROM products;
```

Creating a Real Column with CASE:

```
sql

-- Step 1: Add new column
ALTER TABLE products ADD COLUMN price_tag TEXT;

-- Step 2: Update with CASE
UPDATE products
SET price_tag = CASE
  WHEN price > 1000 THEN 'Expensive'
  WHEN price BETWEEN 500 AND 1000 THEN 'Moderate'
  ELSE 'Cheap'
END;
```

13. Relationships

One-to-One (1:1):

- Each row in Table A is linked to one row in Table B
- Example: Students ↔ Student Profiles
- Student has one profile, profile belongs to one student

```
sql

-- Table 1: students (student_id PRIMARY KEY)
-- Table 2: student_profiles (student_id FOREIGN KEY + UNIQUE)
```

One-to-Many (1:N):

- One row in Table A can be linked to many rows in Table B
- Most commonly used relationship
- Example: Students ↔ Marks
- One student has marks in multiple subjects

```
sql

-- Table 1: students (student_id PRIMARY KEY)
-- Table 2: marks (student_id FOREIGN KEY)
```

Many-to-Many (M:N):

- Many rows in Table A linked to many rows in Table B
- Requires a **junction table** (third table)
- Example: Students ↔ Courses
- One student enrolls in many courses, one course has many students

```
sql

-- Table 1: students (student_id PRIMARY KEY)
-- Table 2: courses (course_id PRIMARY KEY)
-- Table 3: student_courses (student_id FOREIGN KEY, course_id FOREIGN KEY)
-- Composite Primary Key: (student_id, course_id)
```

Junction Table Example:

```
sql

CREATE TABLE student_courses (
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses(course_id),
    PRIMARY KEY (student_id, course_id) -- Composite Primary Key
);
```

14. Joins

Joins combine rows from two or more tables based on a related column.

INNER JOIN:

- Returns only matching rows from both tables

```
sql

SELECT s.name, m.subject, m.marks
FROM students s
INNER JOIN marks m ON s.student_id = m.student_id;
```

LEFT JOIN:

- Returns all rows from left table, and matching rows from right table
- Non-matching rows get NULL values

```
sql

SELECT s.name, m.subject, m.marks
FROM students s
LEFT JOIN marks m ON s.student_id = m.student_id;
```

RIGHT JOIN:

- Returns all rows from right table, and matching rows from left table

```
sql

SELECT s.name, m.subject, m.marks
FROM students s
RIGHT JOIN marks m ON s.student_id = m.student_id;
```

FULL JOIN:

- Returns all rows from both tables
- Non-matching rows get NULL values

```
sql

SELECT s.name, m.subject, m.marks
FROM students s
FULL JOIN marks m ON s.student_id = m.student_id;
```

CROSS JOIN:

- Returns Cartesian product of both tables
- Every row from Table A combined with every row from Table B

```
sql

SELECT * FROM students CROSS JOIN marks;
```

Join Syntax with Aliases:

```
sql

SELECT s.name, m.subject, m.marks
FROM students s
JOIN marks m ON s.student_id = m.student_id;
```

15. Views

A view is a virtual table based on a SQL query.

Benefits:

- Simplify complex queries
- Reuse logic
- Security (expose only selected columns)
- Abstraction (hide table complexity)
- Maintainability

Create View:

```
sql

CREATE VIEW fitness_items AS
SELECT product_id, name, price, stock_quantity
FROM products
WHERE category = 'Fitness';
```

Use View:

```
sql

SELECT * FROM fitness_items;
```

Drop View:

```
sql

DROP VIEW fitness_items;
```

Replace View:

```
sql

CREATE OR REPLACE VIEW fitness_items AS
SELECT product_id, name, price, stock_quantity
FROM products
WHERE category = 'Fitness' AND stock_quantity > 0;
```

16. Stored Procedures

A procedure is a block of SQL code that performs a series of operations.

Benefits:

- Reusability
- Security
- Faster execution
- Encapsulation
- Multi-step operations

Syntax:

```
sql
```

```
CREATE PROCEDURE procedure_name(param1 datatype, param2 datatype)
LANGUAGE plpgsql
AS $$
BEGIN
    -- SQL Logic here
END;
$$;
```

Call Procedure:

```
sql

CALL procedure_name(value1, value2);
```

Example: Add Product Procedure

```
sql

CREATE PROCEDURE add_product(
    p_name VARCHAR,
    p_sku_code CHAR(8),
    p_price NUMERIC,
    p_quantity INT,
    p_category TEXT
)
LANGUAGE plpgsql
AS $$
BEGIN
    INSERT INTO products (name, sku_code, price, stock_quantity, category)
    VALUES (p_name, p_sku_code, p_price, p_quantity, p_category);
    RAISE NOTICE 'Product added successfully';
END;
$$;
```

Call the Procedure:

```
sql

CALL add_product('Bottle', 'B0L12345', 234.00, 45, 'Fitness');
```

17. Common Problems & Solutions

Problem 1: Serial Auto-Increment Issue

- If you manually insert a value into a SERIAL column, the sequence counter doesn't update
- Next auto-insert will conflict

Solution:

```
sql

SELECT last_value FROM product_id_seq; -- Check current value
SELECT SETVAL('product_id_seq', (SELECT MAX(product_id) FROM products));
```

Problem 2: SKU Code Not 8 Characters

- Check constraint ensures length = 8

```
sql
```

```
sku_code CHAR(8) CHECK (LENGTH(sku_code) = 8)
```

Problem 3: Foreign Key Constraint Violation

- Foreign key value must exist in primary key table
- Example: Inserting student_id = 5 when only 1,2,3,4 exist → Error

Solution: Ensure parent record exists first.

Problem 4: NULL Values in NOT NULL Columns

- If column is NOT NULL and no value provided → Error

Solution: Provide value or set DEFAULT.

18. Practice Questions

Clauses & Operators:

1. Show name and price of all products
2. Show all products where category is Electronics
3. Group products by category, show each category once
4. Show categories that have more than one product
5. Show all products sorted by price (ascending/descending)
6. Show only first 3 products
7. Show products with name as "Item Name" and price as "Item Price"
8. Show all unique categories

Aggregation:

1. Total number of products
2. Sum of all prices
3. Average price of all products
4. Minimum price
5. Maximum price
6. Average price of Home & Kitchen and Fitness products
7. Most expensive product in each category

CASE:

1. Create price tag column (Expensive/Moderate/Cheap)
2. Create stock status (In Stock/Out of Stock)
3. Create stock level (High/Medium/Low)

Joins:

1. Show each order with product name and price
2. Show all products even if never ordered

3. Show orders for only Electronics category
4. List all orders sorted by product price
5. Show number of orders placed for each product
6. Show total revenue earned per product
7. Show products where total revenue > 2000
8. Show unique customers who ordered fitness products

Many-to-Many:

1. Show list of students with courses they are enrolled in
2. Find all courses taken by student named Simran

19. Key Takeaways

1. **Database** = Organized collection of data
2. **RDBMS** = Software to manage relational databases
3. **PostgreSQL** = Open-source, feature-rich RDBMS
4. **Tables** = Rows & columns storing data
5. **Constraints** = Rules for data integrity (PRIMARY KEY, NOT NULL, UNIQUE, CHECK, FOREIGN KEY, DEFAULT)
6. **CRUD** = Create, Read, Update, Delete
7. **Clauses** = SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT, AS, DISTINCT
8. **Operators** = Comparison, Logical, Pattern Matching (LIKE), NULL handling
9. **Aggregation** = COUNT, SUM, AVG, MIN, MAX
10. **String Functions** = UPPER, LOWER, LENGTH, SUBSTRING, LEFT, RIGHT, CONCAT, TRIM, REPLACE
11. **CASE** = Conditional logic (IF-ELSE)
12. **Relationships** = One-to-One, One-to-Many, Many-to-Many
13. **Joins** = INNER, LEFT, RIGHT, FULL, CROSS
14. **Views** = Virtual tables from saved queries
15. **Procedures** = Reusable blocks of SQL code

20. Practice Platforms

- **LeetCode SQL** — 50 Essential SQL Questions
- **HackerRank** — SQL challenges (MySQL, Oracle)
- **StrataScratch** — Real-world SQL problems
- **SQLZoo** — Interactive SQL tutorials

Tip: Practice daily. Solve at least 50 questions. Use ChatGPT to generate more practice questions with your table structures.

21. Final Words

- SQL is the most important skill for DBMS exams and interviews

- Queries are asked again and again in placements
- Practice is the only way to master SQL
- Don't memorize — understand the flow
- Use ChatGPT to generate custom practice questions
- Complete the video, practice all exercises, and you'll be confident in SQL

Remember: You are a soldier. You don't back down. You don't break. You move forward together. 🧠

give me interview questions with answers topic wise of all the things covered in this chat

SQL Interview Questions & Answers (Topic-Wise)

Based on the complete transcript, here are topic-wise interview questions with detailed answers.

1. Database Fundamentals

Q1: What is a database?

Answer: A database is an organized collection of data that can be easily accessed, managed, and updated. It stores data electronically in tables with rows and columns, allowing users to query, insert, update, and delete data using a language like SQL.

Q2: What is RDBMS?

Answer: RDBMS stands for Relational Database Management System. It is software that allows you to create, read, update, and delete data using SQL. Data is organized into tables that are related to each other through keys (primary and foreign keys). Examples: PostgreSQL, MySQL, Oracle, SQL Server.

Q3: What is the difference between SQL and NoSQL?

Answer:

SQL	NoSQL
Structured data (tables, rows, columns)	Unstructured data (documents, key-value, graph)
Uses SQL language	Uses various query languages
Fixed schema	Flexible schema
Examples: PostgreSQL, MySQL	Examples: MongoDB, Cassandra
Best for complex relationships	Best for flexible, rapidly changing data

Q4: Why PostgreSQL over MySQL or SQLite?

Answer:

- Open source and free
- Most feature-rich and powerful

- Supports complex queries, joins, CTEs, window functions
- Highly extensible (custom data types, operators)
- Supports NoSQL features (JSON)
- ACID compliant
- Used for enterprise-grade applications

Q5: Why not use Excel instead of a database?

Answer: Excel struggles with big data. When you have 100,000+ rows, searching, updating, and managing data becomes slow. Databases handle millions of rows efficiently with better consistency, reliability, and security. SQL allows complex queries that Excel cannot handle.

2. PostgreSQL Structure

Q6: What is the hierarchy of PostgreSQL?

Answer:

text

Server → Database → Schema → Table → Columns/Rows

- **Database:** Big container storing all data. Multiple databases per server.
- **Schema:** Like a folder inside a database. Organizes tables, views, functions. Default is `public`.
- **Table:** Main place where data is stored in rows and columns.

Q7: What is a schema? Why use multiple schemas?

Answer: A schema is a logical container inside a database that organizes database objects (tables, views, functions). Multiple schemas help organize data by purpose. Example: LinkedIn uses `public` (users, posts), `analytics` (impressions, likes), `admin` (admin profiles), `billing` (payments, subscriptions).

3. CRUD Operations

Q8: What are CRUD operations?

Answer: CRUD stands for Create, Read, Update, Delete.

- **Create:** `CREATE DATABASE`, `CREATE TABLE`, `INSERT INTO`
- **Read:** `SELECT`
- **Update:** `UPDATE`
- **Delete:** `DELETE`

Q9: Write a query to create a table with constraints.

Answer:

sql

```
CREATE TABLE students (  
    student_id SERIAL PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    age INT CHECK (age >= 18),  
    email VARCHAR(100) UNIQUE NOT NULL,  
    grade CHAR(1),  
    created_at DATE DEFAULT CURRENT_DATE  
);
```

Q10: How do you insert multiple rows in one query?

Answer:

```
sql  
  
INSERT INTO students (name, age, grade)  
VALUES ('Akarsh', 23, 'A'),  
       ('Anjali', 22, 'B'),  
       ('Raj', 23, 'C');
```

Q11: How do you update a specific row?

Answer:

```
sql  
  
UPDATE students  
SET age = 24  
WHERE name = 'Akarsh';
```

Q12: How do you delete a specific row?

Answer:

```
sql  
  
DELETE FROM students  
WHERE name = 'Raj';
```

4. Data Types

Q13: What are the numeric data types in PostgreSQL?

Answer:

Type	Size	Range/Use
SMALLINT	2 bytes	-32,768 to 32,767 (age)
INT/INTEGER	4 bytes	-2B to 2B (quantity)
BIGINT	8 bytes	Billions+ (large systems)
DECIMAL/NUMERIC(p,s)	Variable	Exact precision (prices)
REAL	4 bytes	6 decimal digits (ratings)
DOUBLE PRECISION	8 bytes	15 decimal digits

Type	Size	Range/Use
SERIAL	4 bytes	Auto-incrementing integer

Q14: What is the difference between CHAR, VARCHAR, and TEXT?

Answer:

Type	Description
CHAR(n)	Fixed-length, pads with spaces if shorter
VARCHAR(n)	Variable-length with max limit, no padding
TEXT	Unlimited length

Example: CHAR(5) with 'HI' stores 'HI ' (padded). VARCHAR(5) with 'HI' stores 'HI'.

Q15: What is SERIAL? How is it different from INT?

Answer: SERIAL is an auto-incrementing integer. When you insert a row without specifying the value, it automatically increments by 1. INT requires you to provide the value manually. SERIAL is commonly used for primary keys.

Q16: What is NUMERIC(8,2)?

Answer: It means a total of 8 digits, with 2 digits after the decimal point. Maximum value: 999999.99. Used for prices where exact precision is needed.

Q17: What is the difference between DATE, TIME, TIMESTAMP, and TIMESTAMPTZ?

Answer:

Type	Stores	Example
DATE	Only date	2025-07-15
TIME	Only time	14:30:00
TIMESTAMP	Date + time	2025-07-15 14:30:00
TIMESTAMPTZ	Date + time + timezone	2025-07-15 14:30:00+05:30

5. Constraints**Q18: What are constraints? Name all types.**

Answer: Constraints are rules enforced on columns to maintain data integrity. Types:

- **PRIMARY KEY:** Unique + NOT NULL, identifies each row
- **NOT NULL:** Column must have a value
- **UNIQUE:** All values must be different
- **DEFAULT:** Sets default value if none provided
- **CHECK:** Validates data based on condition

- **FOREIGN KEY:** Links to primary key of another table

Q19: Difference between PRIMARY KEY and UNIQUE?

Answer:

PRIMARY KEY	UNIQUE
Cannot be NULL	Can have NULL (unless NOT NULL applied)
Only one per table	Multiple allowed per table
Creates clustered index	Creates non-clustered index
Uniquely identifies each row	Ensures uniqueness only

Q20: What is a FOREIGN KEY? Why is it important?

Answer: A foreign key is a column that references the primary key of another table. It ensures referential integrity — you cannot insert a value that doesn't exist in the parent table. Example: `student_id` in `marks` table references `student_id` in `students` table.

Important Rule: Foreign key values must exist in the primary key table. If primary key has 1,2,3 and foreign key tries to insert 4 → Error.

Q21: What is a CHECK constraint? Give an example.

Answer: CHECK validates data based on a condition.

```
sql

age INT CHECK (age >= 18),
price NUMERIC CHECK (price >= 0),
stock_quantity INT CHECK (stock_quantity >= 0)
```

Q22: Can a column have multiple constraints?

Answer: Yes. Example:

```
sql

email VARCHAR(100) UNIQUE NOT NULL,
age INT CHECK (age >= 18)
```

Q23: What is a composite primary key?

Answer: A primary key made of two or more columns. Used in junction tables for many-to-many relationships.

```
sql

CREATE TABLE student_courses (
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses(course_id),
    PRIMARY KEY (student_id, course_id)
);
```

6. ALTER TABLE

Q24: How do you add a column to an existing table?

Answer:

```
sql

ALTER TABLE students
ADD COLUMN email VARCHAR(100) DEFAULT 'Not Provided';
```

Q25: How do you rename a column?

Answer:

```
sql

ALTER TABLE students
RENAME COLUMN name TO full_name;
```

Q26: How do you change a column's data type?

Answer:

```
sql

ALTER TABLE students
ALTER COLUMN age TYPE SMALLINT;
```

Q27: How do you add and drop a constraint?

Answer:

```
sql

-- Add
ALTER TABLE students
ADD CONSTRAINT age_check CHECK (age >= 0);

-- Drop
ALTER TABLE students
DROP CONSTRAINT age_check;
```

Q28: How do you rename a table?

Answer:

```
sql

ALTER TABLE students
RENAME TO school_students;
```

Q29: How do you set and remove a default value?

Answer:

```
sql

-- Set
ALTER TABLE students
```

```
ALTER COLUMN age SET DEFAULT 18;

-- Remove
ALTER TABLE students
ALTER COLUMN age DROP DEFAULT;
```

7. SQL Clauses

Q30: What are SQL clauses? Name the main ones.

Answer: Clauses are building blocks of SQL queries.

Clause	Purpose
SELECT	Choose columns
FROM	Specify table
WHERE	Filter rows
GROUP BY	Group rows
HAVING	Filter groups
ORDER BY	Sort results
LIMIT	Limit rows
AS	Rename
DISTINCT	Unique values

Q31: Difference between WHERE and HAVING?

Answer:

WHERE	HAVING
Filters rows before grouping	Filters groups after grouping
Cannot use aggregate functions	Can use aggregate functions
Used with SELECT, UPDATE, DELETE	Used with GROUP BY

Example:

```
sql

SELECT category, COUNT(*) FROM products
WHERE price > 100
GROUP BY category
HAVING COUNT(*) > 1;
```

Q32: Difference between GROUP BY and DISTINCT?

Answer:

- **GROUP BY:** Groups rows for aggregation (COUNT, SUM, AVG). Can use aggregate functions.

- **DISTINCT:** Returns only unique values. Cannot use aggregate functions directly.

sql

```
SELECT category FROM products GROUP BY category;  
SELECT DISTINCT category FROM products;
```

Both return unique categories, but GROUP BY is used when you need aggregation.

Q33: What is the order of execution of SQL clauses?

Answer:

1. FROM
2. WHERE
3. GROUP BY
4. HAVING
5. SELECT
6. ORDER BY
7. LIMIT

Q34: What does ORDER BY do? Default order?

Answer: Sorts results. Default is ASC (ascending). Use DESC for descending.

sql

```
SELECT * FROM products ORDER BY price ASC;  -- Low to High  
SELECT * FROM products ORDER BY price DESC; -- High to Low
```

Q35: What is LIMIT?

Answer: Limits the number of rows returned.

sql

```
SELECT * FROM products LIMIT 3;
```

Q36: What is AS? Can it be used for tables?

Answer: AS creates an alias (temporary name). Used for columns and tables.

sql

```
SELECT name AS item_name, price AS item_price FROM products;  
SELECT s.name FROM students AS s;
```

8. Operators

Q37: What are comparison operators?

Answer:

Operator	Meaning
=	Equal
!= or <>	Not equal
<	Less than
>	Greater than
<=	Less than or equal
>=	Greater than or equal

Q38: What are logical operators?

Answer:

- **AND:** All conditions must be true
- **OR:** Any condition can be true
- **NOT:** Reverses the condition

Q39: What is BETWEEN? How is it different from >= and <=?

Answer: BETWEEN checks if a value is within a range (inclusive).

```
sql

WHERE price BETWEEN 400 AND 1000;
-- Same as: WHERE price >= 400 AND price <= 1000;
```

BETWEEN is a shortcut for multiple AND conditions.

Q40: What is IN operator?

Answer: IN checks if a value matches any value in a list.

```
sql

WHERE category IN ('Electronics', 'Fitness', 'Home & Kitchen');
-- Same as: WHERE category = 'Electronics' OR category = 'Fitness' OR category = 'Home & Kitchen';
```

Q41: Explain LIKE with wildcards.

Answer:

Pattern	Meaning
LIKE 'W%'	Starts with W
LIKE '%W'	Ends with W
LIKE '%W%'	Contains W
LIKE '_W%'	Second letter is W
LIKE '__W%'	Third letter is W

% = any number of characters, _ = exactly one character.

Q42: How do you handle NULL values?

Answer:

```
sql

WHERE column IS NULL;
WHERE column IS NOT NULL;
```

You cannot use = NULL — it won't work. Always use IS NULL or IS NOT NULL .

9. Aggregation Functions

Q43: What are aggregation functions? Name them.

Answer: Functions that summarize data.

Function	Purpose
COUNT()	Count rows
SUM()	Add values
AVG()	Average
MIN()	Smallest
MAX()	Largest
ROUND()	Round decimals

Q44: Difference between COUNT(*) and COUNT(column)?

Answer:

- COUNT(*) counts all rows including NULLs.
- COUNT(column) counts only non-NULL values in that column.

Q45: Can you use aggregation functions without GROUP BY?

Answer: Yes, if you want a single result for the entire table.

```
sql

SELECT COUNT(*) FROM products;
SELECT AVG(price) FROM products;
```

If you mix aggregate and non-aggregate columns without GROUP BY, you get an error.

Q46: How do you round an average to 2 decimal places?

Answer:

```
sql
```

```
SELECT ROUND(AVG(price), 2) FROM products;
```

Q47: What is the difference between COUNT and SUM?

Answer:

- COUNT counts number of rows/values.
- SUM adds numeric values.

sql

```
SELECT COUNT(product_id), SUM(price) FROM products;
```

10. String Functions

Q48: Name common string functions in PostgreSQL.

Answer:

Function	Purpose
UPPER()	Uppercase
LOWER()	Lowercase
LENGTH()	Length
SUBSTRING()	Extract part
LEFT()	Extract from left
RIGHT()	Extract from right
CONCAT()	Join strings
CONCAT_WS()	Join with separator
TRIM()	Remove spaces
REPLACE()	Replace text

Q49: Difference between CONCAT and CONCAT_WS?

Answer:

- CONCAT(a, b) joins without separator: CONCAT('Hello', 'World') → 'HelloWorld'
- CONCAT_WS(sep, a, b) joins with separator: CONCAT_WS(' ', 'Hello', 'World') → 'Hello World'

Q50: How do you extract first 2 characters from a string?

Answer:

sql

```
SELECT LEFT(sku_code, 2) FROM products;  
-- or
```

```
SELECT SUBSTRING(sku_code, 1, 2) FROM products;
```

Q51: How do you replace a substring?

Answer:

```
sql

SELECT REPLACE(sku_code, 'WM', 'GG') FROM products;
```

Q52: What does TRIM do?

Answer: Removes leading and trailing spaces.

```
sql

SELECT TRIM('  hello  '); -- Returns 'heLlo'
```

11. CASE Statements

Q53: What is CASE in SQL?

Answer: CASE is a conditional expression (like IF-ELSE). It returns different values based on conditions.

```
sql

SELECT name, price,
       CASE
         WHEN price > 1000 THEN 'Expensive'
         WHEN price BETWEEN 500 AND 1000 THEN 'Moderate'
         ELSE 'Cheap'
       END AS price_tag
FROM products;
```

Q54: Can CASE be used in UPDATE?

Answer: Yes.

```
sql

UPDATE products
SET price_tag = CASE
  WHEN price > 1000 THEN 'Expensive'
  WHEN price BETWEEN 500 AND 1000 THEN 'Moderate'
  ELSE 'Cheap'
END;
```

Q55: Can CASE be used without ELSE?

Answer: Yes, but non-matching rows get NULL. ELSE is optional but recommended.

Q56: Difference between CASE and IF-ELSE?

Answer: CASE is SQL's version of IF-ELSE. It works inside a query and returns a value. IF-ELSE is procedural (used in functions/procedures).

12. Relationships

Q57: What are the types of relationships in SQL?

Answer:

1. **One-to-One (1:1):** Each row in A linked to one row in B. Example: Students ↔ Profiles.
2. **One-to-Many (1:N):** One row in A linked to many rows in B. Example: Students ↔ Marks.
3. **Many-to-Many (M:N):** Many rows in A linked to many rows in B. Requires junction table. Example: Students ↔ Courses.

Q58: How do you implement One-to-One?

Answer: Add a foreign key in one table that references the primary key of the other table, with a UNIQUE constraint.

```
sql

CREATE TABLE student_profiles (
    student_id INT PRIMARY KEY REFERENCES students(student_id),
    address TEXT,
    phone VARCHAR(15)
);
```

Q59: How do you implement One-to-Many?

Answer: Add a foreign key in the "many" table referencing the primary key of the "one" table.

```
sql

CREATE TABLE marks (
    mark_id SERIAL PRIMARY KEY,
    student_id INT REFERENCES students(student_id),
    subject VARCHAR(50),
    marks INT
);
```

Q60: How do you implement Many-to-Many?

Answer: Create a junction table with foreign keys to both tables and a composite primary key.

```
sql

CREATE TABLE student_courses (
    student_id INT REFERENCES students(student_id),
    course_id INT REFERENCES courses(course_id),
    PRIMARY KEY (student_id, course_id)
);
```

Q61: What is a junction table?

Answer: A table that connects two tables in a many-to-many relationship. It contains foreign keys to both tables. Example: student_courses connects students and courses .

13. Joins

Q62: What is a JOIN? Why use it?

Answer: JOIN combines rows from two or more tables based on a related column. Used to merge related data spread across multiple tables.

Q63: Types of JOINS?

Answer:

- **INNER JOIN:** Only matching rows
- **LEFT JOIN:** All rows from left + matching from right
- **RIGHT JOIN:** All rows from right + matching from left
- **FULL JOIN:** All rows from both
- **CROSS JOIN:** Cartesian product

Q64: Difference between INNER JOIN and LEFT JOIN?

Answer:

- **INNER JOIN:** Returns only rows where both tables have matching values.
- **LEFT JOIN:** Returns all rows from left table, with NULLs for non-matching right table rows.

Example: Students who took exams vs all students (including those who didn't).

Q65: What is CROSS JOIN?

Answer: Returns Cartesian product — every row from Table A combined with every row from Table B. No ON condition needed.

```
sql

SELECT * FROM students CROSS JOIN marks;
```

If A has 5 rows and B has 4 rows, result has 20 rows.

Q66: Write an INNER JOIN query.

Answer:

```
sql

SELECT s.name, m.subject, m.marks
FROM students s
INNER JOIN marks m ON s.student_id = m.student_id;
```

Q67: How do you find products never ordered?

Answer: Use LEFT JOIN and check for NULL.

```
sql

SELECT p.name, o.order_id
FROM products p
```

```
LEFT JOIN orders o ON p.product_id = o.product_id
WHERE o.order_id IS NULL;
```

Q68: Can you join more than two tables?

Answer: Yes.

```
sql

SELECT s.name, c.course_name
FROM students s
JOIN student_courses sc ON s.student_id = sc.student_id
JOIN courses c ON sc.course_id = c.course_id;
```

Q69: What is a self-join?

Answer: Joining a table with itself. Used for hierarchical data (e.g., employee-manager).

```
sql

SELECT e.name AS employee, m.name AS manager
FROM employees e
JOIN employees m ON e.manager_id = m.employee_id;
```

14. Views

Q70: What is a view?

Answer: A view is a virtual table based on a SQL query. It doesn't store actual data but shows results when accessed. Like a saved query.

Q71: Why use views?

Answer:

- Simplify complex queries
- Reuse logic
- Security (expose only selected columns)
- Abstraction (hide table complexity)
- Maintainability (update logic in one place)

Q72: How do you create and use a view?

Answer:

```
sql

CREATE VIEW fitness_items AS
SELECT product_id, name, price, stock_quantity
FROM products
WHERE category = 'Fitness';

SELECT * FROM fitness_items;
```

Q73: How do you drop a view?

Answer:

```
sql

DROP VIEW fitness_items;
```

Q74: Can you update data through a view?

Answer: Simple views (single table, no aggregates) can be updatable. Complex views (joins, aggregates) are usually read-only.

Q75: Difference between a view and a table?

Answer:

View	Table
Virtual (no data stored)	Stores actual data
Based on query	Physical structure
Read-only mostly	Read/write
No indexes directly	Can have indexes

15. Stored Procedures

Q76: What is a stored procedure?

Answer: A stored procedure is a block of SQL code that performs a series of operations (insert, update, delete, select) and is stored in the database. Like a function in programming.

Q77: Benefits of stored procedures?

Answer:

- Reusability (write once, use many times)
- Security (logic stored in DB, no direct table access)
- Faster execution (compiled and stored on server)
- Encapsulation (hide complex logic)
- Multi-step operations in one call

Q78: How do you create and call a procedure?

Answer:

```
sql

CREATE PROCEDURE add_product(
    p_name VARCHAR,
    p_sku_code CHAR(8),
    p_price NUMERIC,
    p_quantity INT,
    p_category TEXT
)
LANGUAGE plpgsql
```

```
AS $$
BEGIN
    INSERT INTO products (name, sku_code, price, stock_quantity, category)
    VALUES (p_name, p_sku_code, p_price, p_quantity, p_category);
    RAISE NOTICE 'Product added successfully';
END;
$$;

CALL add_product('Bottle', 'BOL12345', 234.00, 45, 'Fitness');
```

Q79: Difference between a function and a procedure?

Answer:

Function	Procedure
Must return a value	May or may not return
Called in SELECT	Called with CALL
Cannot do transactions	Can do transactions
Used for calculations	Used for operations

Q80: What is PL/pgSQL?

Answer: Procedural Language for PostgreSQL. It extends SQL with programming features (variables, loops, conditions) for writing functions and procedures.

16. Common Problems & Solutions

Q81: What happens if you manually insert into a SERIAL column?

Answer: The sequence counter doesn't update. Next auto-insert will conflict.
Solution:

```
sql

SELECT SETVAL('product_id_seq', (SELECT MAX(product_id) FROM products));
```

Q82: How do you ensure SKU code is exactly 8 characters?

Answer:

```
sql

sku_code CHAR(8) CHECK (LENGTH(sku_code) = 8)
```

Q83: What is a foreign key constraint violation?

Answer: When you try to insert a foreign key value that doesn't exist in the parent table.
Solution: Ensure parent record exists first.

Q84: How do you handle NULL in NOT NULL columns?

Answer: Either provide a value or set a DEFAULT.

```
sql
```

```
name VARCHAR(50) NOT NULL,  
created_at DATE DEFAULT CURRENT_DATE
```

17. Query Writing (Practical)

Q85: Write a query to find the cheapest product.

Answer:

```
sql
```

```
SELECT name, price FROM products  
WHERE price = (SELECT MIN(price) FROM products);
```

Q86: Write a query to find the most expensive product in each category.

Answer:

```
sql
```

```
SELECT category, MAX(price) AS max_price  
FROM products  
GROUP BY category;
```

Q87: Write a query to count orders per product.

Answer:

```
sql
```

```
SELECT p.name, COUNT(o.order_id) AS total_orders  
FROM products p  
LEFT JOIN orders o ON p.product_id = o.product_id  
GROUP BY p.name;
```

Q88: Write a query to find total revenue per product.

Answer:

```
sql
```

```
SELECT p.name, SUM(o.quantity * p.price) AS revenue  
FROM products p  
JOIN orders o ON p.product_id = o.product_id  
GROUP BY p.name;
```

Q89: Write a query to show products with revenue > 2000.

Answer:

```
sql
```

```
SELECT p.name, SUM(o.quantity * p.price) AS revenue  
FROM products p  
JOIN orders o ON p.product_id = o.product_id
```

```
GROUP BY p.name
HAVING SUM(o.quantity * p.price) > 2000;
```

Q90: Write a query to find unique customers who ordered fitness products.

Answer:

```
sql

SELECT DISTINCT o.customer_name
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE p.category = 'Fitness';
```

Q91: Write a query to show all products even if never ordered.

Answer:

```
sql

SELECT p.name, o.order_id
FROM products p
LEFT JOIN orders o ON p.product_id = o.product_id;
```

Q92: Write a query to show orders only for Electronics category.

Answer:

```
sql

SELECT o.order_id, p.name, p.category
FROM orders o
JOIN products p ON o.product_id = p.product_id
WHERE p.category = 'Electronics';
```

Q93: Write a query to create a price tag column.

Answer:

```
sql

SELECT name, price,
       CASE
         WHEN price > 1000 THEN 'Expensive'
         WHEN price BETWEEN 500 AND 1000 THEN 'Moderate'
         ELSE 'Cheap'
       END AS price_tag
FROM products;
```

Q94: Write a query to create a stock status column.

Answer:

```
sql

SELECT name,
       CASE
         WHEN is_available THEN 'In Stock'
         ELSE 'Out of Stock'
       END AS stock_status
FROM products;
```

```
END AS stock_status  
FROM products;
```

Q95: Write a query to find students and their courses (many-to-many).

Answer:

```
sql  
  
SELECT s.name, c.course_name  
FROM students s  
JOIN student_courses sc ON s.student_id = sc.student_id  
JOIN courses c ON sc.course_id = c.course_id;
```

18. Conceptual Questions

Q96: What is the difference between DELETE, TRUNCATE, and DROP?

Answer:

DELETE	TRUNCATE	DROP
Removes specific rows	Removes all rows	Removes table structure
Can use WHERE	Cannot use WHERE	Removes entire table
Rollback possible	Rollback possible	Cannot rollback
Slower	Faster	Fastest
Triggers fire	Triggers don't fire	Triggers don't fire

Q97: What is a primary key vs foreign key?

Answer:

Primary Key	Foreign Key
Uniquely identifies each row	Links to primary key of another table
Cannot be NULL	Can be NULL
Only one per table	Multiple allowed
Creates clustered index	Creates non-clustered index

Q98: What is normalization?

Answer: Organizing data to reduce redundancy and improve integrity. Normal forms: 1NF, 2NF, 3NF, BCNF. Splitting data into multiple related tables.

Q99: What is denormalization?

Answer: Adding redundancy back to improve read performance. Used in data warehousing.

Q100: What is an index?

Answer: A data structure that speeds up data retrieval. Like a book's index. Created on columns used in WHERE, JOIN, ORDER BY.

```
sql

CREATE INDEX idx_product_name ON products(name);
```

Q101: What is a transaction?

Answer: A sequence of operations that must all succeed or all fail. ACID properties: Atomicity, Consistency, Isolation, Durability.

```
sql

BEGIN;
UPDATE accounts SET balance = balance - 100 WHERE id = 1;
UPDATE accounts SET balance = balance + 100 WHERE id = 2;
COMMIT;
```

Q102: What is ACID?

Answer:

- **Atomicity:** All or nothing
- **Consistency:** Data remains valid
- **Isolation:** Concurrent transactions don't interfere
- **Durability:** Committed data persists

Q103: What is a subquery?

Answer: A query inside another query.

```
sql

SELECT name FROM products
WHERE price = (SELECT MAX(price) FROM products);
```

Q104: What is a correlated subquery?

Answer: A subquery that references the outer query. Runs for each row of outer query.

```
sql

SELECT name FROM products p
WHERE price > (SELECT AVG(price) FROM products WHERE category = p.category);
```

Q105: What is a CTE (Common Table Expression)?

Answer: A temporary result set defined with WITH.

```
sql
```

```
WITH expensive_products AS (  
    SELECT * FROM products WHERE price > 1000  
)  
SELECT * FROM expensive_products;
```

Q106: What is a window function?

Answer: Performs calculation across a set of rows related to current row.

```
sql  
  
SELECT name, price,  
       RANK() OVER (ORDER BY price DESC) AS rank  
FROM products;
```

Q107: What is the difference between RANK, DENSE_RANK, and ROW_NUMBER?

Answer:

- **ROW_NUMBER:** Sequential number (1,2,3,4)
- **RANK:** Same rank for ties, skips next (1,2,2,4)
- **DENSE_RANK:** Same rank for ties, no skip (1,2,2,3)

Q108: What is a trigger?

Answer: A function that automatically executes when an event (INSERT, UPDATE, DELETE) occurs on a table.

```
sql  
  
CREATE TRIGGER update_timestamp  
BEFORE UPDATE ON products  
FOR EACH ROW  
EXECUTE FUNCTION update_last_updated();
```

Q109: What is a self-join? Give an example.

Answer: Joining a table with itself.

```
sql  
  
SELECT e.name AS employee, m.name AS manager  
FROM employees e  
JOIN employees m ON e.manager_id = m.employee_id;
```

Q110: What is the difference between UNION and UNION ALL?

Answer:

- **UNION:** Removes duplicates
- **UNION ALL:** Keeps duplicates (faster)

```
sql  
  
SELECT name FROM students  
UNION  
SELECT name FROM teachers;
```

Quick Reference Cheat Sheet

Topic	Key Points
Database	Organized collection of data
RDBMS	Software to manage relational DB
PostgreSQL	Open-source, feature-rich RDBMS
Schema	Folder inside database
Table	Rows & columns
CRUD	Create, Read, Update, Delete
Constraints	PRIMARY KEY, NOT NULL, UNIQUE, DEFAULT, CHECK, FOREIGN KEY
Data Types	INT, VARCHAR, CHAR, TEXT, DATE, TIMESTAMP, BOOLEAN, NUMERIC
Clauses	SELECT, FROM, WHERE, GROUP BY, HAVING, ORDER BY, LIMIT
Operators	=, !=, <, >, AND, OR, NOT, LIKE, IN, BETWEEN, IS NULL
Aggregation	COUNT, SUM, AVG, MIN, MAX, ROUND
String Functions	UPPER, LOWER, LENGTH, SUBSTRING, LEFT, RIGHT, CONCAT, TRIM, REPLACE
CASE	IF-ELSE logic
Relationships	1:1, 1:N, M:N
Joins	INNER, LEFT, RIGHT, FULL, CROSS
Views	Virtual table from query
Procedures	Reusable block of SQL
Transactions	BEGIN, COMMIT, ROLLBACK
ACID	Atomicity, Consistency, Isolation, Durability

Final Tips for Interviews

1. **Practice writing queries** on paper/whiteboard without IDE
2. **Understand execution order** of clauses
3. **Know the difference** between similar concepts (WHERE vs HAVING, DELETE vs TRUNCATE vs DROP)
4. **Be ready to explain** your query logic step by step
5. **Practice on LeetCode, HackerRank, StrataScratch**
6. **Use ChatGPT** to generate more practice questions
7. **Don't memorize** — understand the flow
8. **Always handle NULLs** in your queries

9. **Use aliases** for readability
10. **Test edge cases** (empty results, NULLs, duplicates)

Remember: You are a soldier. You don't back down. You don't break. You move forward together. 🌀